

Phase 0 Implementation Summary

AI-Powered GitHub Repository Modernization Platform

Scope: Research, architecture, local-first monorepo foundation, development tooling, shared types, and database foundation.

Status: Phase 0 implementation is mostly complete; final database migration, Ollama verification, validation commands, and Phase 0 commit should be completed before Phase 1 starts.

Area	Status	Summary
Package manager	Done	pnpm workspace configured and working after bypassing Corepack issues.
Monorepo tooling	Done	Turborepo, TypeScript, ESLint, Prettier, Husky, lint-staged, Vitest, and Playwright installed/configured.
Core package	Done	packages/core-types created as the shared type foundation.
Database package	In progress	Prisma v7 + PostgreSQL setup is being finalized in packages/db.
Local services	Pending verify	PostgreSQL Docker and Ollama endpoint still need final validation.
Phase 1	Not started	No GitHub Repository Discovery implementation has started yet.

1. Phase 0 Objective

Phase 0 establishes the technical foundation for the platform before any GitHub repository discovery, scanning, rule engine, or AI workflow is implemented. The goal is to make the repository ready for structured development with repeatable tooling, predictable package management, a shared type system, local database setup, and local AI runtime verification.

Important boundary: Phase 1 has not started. The work completed so far belongs to Phase 0 only.

2. Repository Foundation Created

- Created the pnpm workspace layout with apps/*, packages/*, tools/*, and examples/* support.
- Established root-level project metadata and scripts in package.json.
- Configured the project as a private TypeScript monorepo.
- Added package manager enforcement to prevent accidental npm/yarn usage in the project.
- Added .gitignore rules for node_modules, build outputs, test outputs, env files, database files, Prisma local artifacts, Docker runtime data, and secrets.

3. Package Manager and Monorepo Tooling

The project initially hit Corepack-related pnpm issues. Corepack was bypassed and pnpm was installed globally through npm. After that, pnpm was verified and the workspace lockfile was generated successfully.

Tool	Purpose	Implementation status
pnpm	Workspace package manager	Installed and used as the only package manager.
Turborepo	Build orchestration and caching	Configured with dev, build, typecheck, lint, and test tasks.
TypeScript	Strict typing across apps/packages	Root base config and package-level configs created.
ESLint	Code quality checks	Flat eslint.config.js created for ESLint v9+ style configuration.
Prettier	Consistent formatting	prettier.config.js and .prettierrignore created.
Husky	Git hook management	pre-commit hook configured to run lint-staged.
lint-staged	Checks staged files only	Configured for JS/TS and common config/document files.
Vitest	Unit/integration testing	Root script and package script setup included.
Playwright	Dashboard E2E testing	Installed for later dashboard phase; not actively used yet.

4. Root Configuration Files Added

File	Purpose
pnpm-workspace.yaml	Defines workspace packages under apps/*, packages/*, tools/*, and examples/.*
package.json	Root scripts, package manager, dev tooling, and workspace metadata.
turbo.json	Defines Turborepo task graph and output caching rules.
tsconfig.base.json	Shared strict TypeScript compiler settings.
tsconfig.json	Root TypeScript project entry point for apps, packages, and tooling files.
eslint.config.js	ESLint flat config for TypeScript and project-wide linting.
prettier.config.js	Project formatting rules.
.prettierrignore	Excludes generated/dependency files from formatting.
lint-staged.config.js	Runs ESLint and Prettier on staged files before commit.
.husky/pre-commit	Runs pnpm exec lint-staged before each commit.
docker-compose.yml	Local PostgreSQL service for development.
.env.example	Example DATABASE_URL and OLLAMA_HOST configuration.

5. Shared Core Types Package

Created packages/core-types as the foundation package for shared TypeScript types used across future modules such as github-client, repo-scanner, stack-detector, rule-engine, api, and dashboard.

Implemented files

File	Purpose
packages/core-types/package.json	Declares @repo/core-types as an internal workspace package.
packages/core-types/tsconfig.json	Build configuration for the core-types package.
packages/core-types/src/index.ts	Exports initial shared types such as Severity and RepoMetadata.

Why this package exists

- It prevents duplicate type definitions across packages.
- It gives every package one source of truth for shared domain models.
- It allows future packages to import stable types from @repo/core-types.
- It also gives TypeScript a real package to compile during Phase 0 validation.

6. Database Foundation

A database foundation was added for repository metadata, findings, stack profiles, health scores, and run tracking. The setup moved toward Prisma v7 conventions with PostgreSQL and a JavaScript driver adapter.

Component	Implementation summary
docker-compose.yml	Defines local PostgreSQL service with a named persistent Docker volume.
.env.example	Defines DATABASE_URL and OLLAMA_HOST values for local development.
packages/db	Created or planned as the internal database package.
Prisma v7	Configuration moved to prisma.config.ts; schema datasource keeps provider only.
@prisma/adapters-pg + pg	Used for Prisma v7 PostgreSQL connection via JavaScript driver adapter.

Prisma v7 decisions made

- schema.prisma should not contain url = env("DATABASE_URL").
- packages/db/prisma.config.ts should load dotenv and provide datasource.url dynamically.
- Prisma CLI and @prisma/client should live inside packages/db for pnpm workspace correctness.
- packages/db/src/index.ts should use pg.Pool and @prisma/adapters-pg to initialize PrismaClient.

7. Issues Resolved During Phase 0

Issue	Resolution
Corepack pnpm key/signature failure	Bypassed Corepack and installed pnpm globally using npm.
npm used inside pnpm-only workspace	Stopped npm commands inside the project and used pnpm only.
pnpm root dependency warning	Used -w for root-level dev dependencies.
Husky PATH / bad Node command	Identified broken npm-installed Node path and verified correct Node path.
ESLint config missing	Created eslint.config.js using ESLint flat config.
Prettier CJS warning	Changed from prettier.config.cjs to prettier.config.js because root uses type: module.
TypeScript no-input warning	Created packages/core-types/src/index.ts.
Turborepo packageManager warning	Aligned packageManager and devEngines.packageManager versions.
Prisma v7 behavior changes	Adjusted schema/config/client approach for Prisma v7.

8. Validation Progress

The tooling has been partially validated. The following commands were run or reached successful states during setup:

Command	Observed status
pnpm -v	Confirmed pnpm 11.8.0.
pnpm install	Completed after package manager and build-script approval fixes.
pnpm format:check	Formatting issues detected first, then resolved with pnpm format.
pnpm lint	Ran successfully after eslint.config.js was added.
pnpm typecheck	Succeeded for @repo/core-types after packageManager fix.
pnpm exec lint-staged	Successfully prepared, ran staged tasks, and staged formatting changes.

9. Remaining Phase 0 Closure Tasks

Before Phase 0 is considered fully complete, run and verify the remaining closure checklist below.

Task	Required action
Prisma build approval	Run pnpm approve-builds if pnpm reports ignored Prisma build scripts.
Database package	Finalize packages/db package.json, prisma.config.ts, schema.prisma, and src/index.ts.
PostgreSQL	Run docker compose up -d when migration is needed.
Prisma generate	Run pnpm --filter @repo/db db:generate.
Prisma migration	Run pnpm --filter @repo/db db:migrate and create init migration.
Ollama	Verify curl http://localhost:11434/api/tags returns model JSON.
Final validation	Run pnpm format:check, pnpm lint, pnpm typecheck, pnpm build, pnpm test, and pnpm exec lint-staged.
Commit	Create the Phase 0 commit: chore: complete phase 0 foundation.

10. Phase 0 Exit Criteria

- docs/scope.md exists and defines goal, problem statement, success metrics, non-goals, and constraints.
- docs/architecture.md exists and documents monorepo layout, dependency direction, packages, local AI runtime, and database design.
- Monorepo builds with pnpm build.
- TypeScript typecheck passes with pnpm typecheck.
- ESLint passes with pnpm lint.
- Prettier passes with pnpm format:check.
- Husky/lint-staged runs successfully before commit.
- Prisma client generation and initial migration succeed against local PostgreSQL.
- Ollama local endpoint responds successfully.
- A clean Git commit exists for Phase 0 foundation.

Next gate: After these exit criteria pass, ask for confirmation before starting Phase 1 - Repository Discovery.

Appendix: Current Phase 0 Directory Shape

```
ai_powered_github_repository_modernization_platform/  
  apps/  
  docs/  
  packages/  
    core-types/  
    db/ planned/finalizing  
  .husky/  
  .gitignore  
  .prettierignore  
  docker-compose.yml  
  .env.example  
  eslint.config.js  
  lint-staged.config.js  
  package.json  
  pnpm-lock.yaml  
  pnpm-workspace.yaml  
  prettier.config.js  
  tsconfig.base.json  
  tsconfig.json  
  turbo.json
```